🚨 GRADE BOOSTER: ALERTS & SPECIFICATIONS

💀 Exam Traps & "Distinction" Nuances

- **The "Quiz Week" Confusion:** The quiz is **NOT** in Week 3. It is in **Week 4** (covering content from Weeks 1-3).
  - *Format:* 5% Theory (MCQ, Closed Book) + 80% Practical (Coding, Open Book/Cheat Sheet allowed).
- **The "Hart" Correction:** The professor explicitly corrected a previous statement. A **Hart** is a **Hardware Thread** (essentially a CPU core in the simulator). In xv6, Hart 0 usually handles initialization, then wakes up other Harts.
- **User vs. Kernel Stack Trap:**
  - *The Trap:* Students often think a process only has one stack.
  - *The Reality:* Every process has **two** stacks: a User Stack (for app code) and a Kernel Stack (for when it traps into the OS).
  - *Why?* If we used the user stack while in the kernel, we might corrupt user data, or worse, if we context switch, we wouldn't have a reliable place to store the kernel's execution state for that process.
- **Linux vs. Windows Argument Passing:**
  - *Nuance:* In x64, Windows spills extra arguments to the stack. Linux (System V ABI) strictly prefers registers and doesn't support spilling extra args directly for syscalls (you'd pass a struct pointer instead).

🛠 Lab & Practical Quiz Intel

- **Gradescope is King:** Do not rely on local execution only. Your code must compile and run on the Gradescope environment (Ubuntu 22.04 compat).
- **Cheat Sheet Strategy:** You are allowed a cheat sheet for the **Practical Quiz**.
  - *Recommended content:* The step-by-step flow of adding a system call (files to modify), basic `printf` usage in kernel (`printf` is safe in kernel, `std::cout` is not), and the register mapping for RISC-V (`a0` - `a7`).
- **Adding Syscalls (Lab 1):** You will likely be tested on implementing a simple syscall. Memorize the file chain: `syscall.h` -> `syscall.c` -> `sysproc.c` -> `usys.pl` -> `user.h` -> `Makefile` .

📖 LECTURE RECONSTRUCTION

1. The Kernel's Job & Security Models

Concept: The OS is a Resource Allocator and a Security Enforcer.
The Lecture Deep-Dive:
Security isn't just passwords; it's **isolation**. The OS assumes the user is "evil."

- **Protection mechanism:** We use hardware modes (User Mode vs. Kernel Mode).
- **Privileged Instructions:** Instructions that can crash the system or spy on others (e.g., disabling interrupts, changing page tables `satp`, accessing control registers `sstatus`) are banned in User Mode.
- **Attack Vector Example:** Buffer Overflow.
  - *Scenario:* User writes past the end of an array into the stack.
  - *Goal:* Overwrite the Return Address (PC) to point to malicious code injected into the stack.
  - *Defense:* Mark the stack memory as **Non-Executable**. If the PC jumps there, the hardware raises an exception.

2. System Calls (The Gateway)

Definition: A controlled entry point allowing user programs to request kernel services (like `read`, `write`, `fork`).

Mechanism (RISC-V):
We do **not** call kernel functions directly (like `call function`). We use a special instruction: `ecall`.

- **Register `a7`:** Holds the System Call Number (ID).
- **Registers `a0` - `a6`:** Hold the arguments.
- **Return Value:** Placed in `a0` after the call.

Comparison:

- **Direct Call:** Unsafe. User code and Kernel code share the same memory/privilege.
- **API (POSIX/Win32):** We use APIs (like C standard library) instead of raw syscalls for:
  1. **Portability:** Code runs on Mac/Linux without changing.
  2. **Efficiency:** Libraries buffer data (read large chunks instead of 1 byte at a time).
  3. **Simplicity:** APIs handle the complex register setup.

3. Traps, Exceptions, and Interrupts

Concept: Events that force the CPU to stop user code and jump to the kernel.

Type	Initiated By	Example	Predictable?
System Call	User Program	`ecall` (File I/O)	Yes (Intentional)
Exception	Hardware (Error)	Division by Zero, Page Fault	Yes (Synchronous)
Interrupt	Hardware (Device)	Disk ready, Timer tick	No (Asynchronous)

The "Transparent Resume" Rule:
When a trap occurs, the OS must save the *exact* state of the user program. When the trap finishes, the user program must resume as if nothing happened.

4. The xv6 Trap Handling Flow (Critical Path)

This is the core mechanical process of the OS. Master this.

Phase 1: The Trigger (Hardware)

1. CPU detects `ecall` or Interrupt.
2. **Switch Mode:** User Mode → Kernel Mode.
3. **Save PC:** Current user instruction address is saved to `sepc` (Supervisor Exception Program Counter).
4. **Disable Interrupts:** To prevent data corruption during the switch (`sstatus.SIE = 0`).
5. **Jump:** CPU jumps to the address in `stvec` (the Trampoline).

Phase 2: The Trampoline (`trampoline.S`)
Context: We are in Kernel Mode, but we are still using the **User's Page Table**. We cannot easily access kernel data yet.

1. **Swap `a0` and `sscratch`:** `csrw sscratch, a0`. We need a register to work with, but we can't overwrite user data. `sscratch` holds the address of the `trapframe`.
2. **Save User State:** Save all 32 user registers into the `trapframe` (which is mapped in user space).
3. **Load Kernel State:** Load the Kernel Stack Pointer and Kernel Page Table address from the `trapframe`.
4. **Fence:** Execute `sfence.vma` to flush old memory mappings.
5. **Jump to C Code:** Call `usertrap()`.

Phase 3: The Handler (`trap.c`)

1. Check `scause` register: Is it a syscall (8)? An interrupt?
2. If Syscall:
   - Retrieve syscall number from saved `a7`.
   - Dispatch to specific function (e.g., `sys_write`).
   - Store return value in saved `a0`.
   - **Crucial:** Increment saved `epc` by 4 (to skip the `ecall` instruction when returning).

Phase 4: The Return (`trampoline.S`)

1. Restore user registers from `trapframe`.
2. Execute `sret` (Supervisor Return).
   - Copies `sepc` back to `pc`.
   - Switches Privilege Mode back to User.

- Re-enables interrupts.

5. Adding a System Call (Implementation Guide)

For the Practical Quiz, memorize this checklist.

1. `kernel/syscall.h`: Add `#define SYS_hello 22` (Use the next available number).
2. `kernel/syscall.c`:
   - Add `extern uint64 sys_hello(void);`
   - Add `[SYS_hello] sys_hello,` to the function pointer array.
3. `kernel/sysproc.c`: Write the actual C implementation:

```c
uint64 sys_hello(void) {
    printf("Hello from kernel!\n");
    return 0;
}
```

4. `kernel/defs.h`: Add the function prototype so other kernel files can see it.
5. `user/usys.pl`: Add `entry("hello");`. This generates the assembly stub that puts the ID in `a7` and calls `ecall`.
6. `user/user.h`: Add the prototype `int hello(void);` so user programs can call it.
7. `Makefile`: Add `_hello` to `UPROGS` to compile your test program.

🗣 Q&A CORNER

- **Student Q:** Why can't we just use `sudo` to access kernel mode?
- **Prof A:** `sudo` is a software concept (OS permissions). It does **not** put the CPU into Kernel Mode physically. Even running as root/sudo, your shell is still executing in User Mode (Ring 3). Only the `ecall` or interrupt switches the hardware execution mode.
- **Student Q:** Why do we need the `sscratch` register in the trampoline?
- **Prof A:** When we first trap, we need to save the user's registers to memory (the trapframe). But to write to memory, we need a register to hold the address! If we use a register (like `a0`) to hold the address, we destroy the user's data in `a0`. `sscratch` is a special hardware register used to swap values with `a0` atomically, saving the user's data while giving us a working register.
- **Student Q:** Can we modify the `trapframe`?
- **Prof A:** The `trapframe` is mapped into the user's virtual address space, but it is **Read-Only** for the user. If the user could write to it, they could overwrite the kernel stack pointer stored there and hijack the kernel upon the next trap.

⚡ EXECUTIVE SUMMARY (TL;DR)

1. **Assessment:** Week 4 Quiz = 80% Coding (Open Book), 5% Theory (Closed Book). Focus on Lab 1 flow.
2. **System Calls:** Use `ecall`. ID in `a7`. Args in `a0` - `a6`.
3. **Traps:** 3 Types (Syscall, Exception, Interrupt). All handled via `trampoline.S`.
4. **Trapframe:** The "save game" state for a process. Stores user registers when in kernel, and kernel context when in user mode.
5. **Two Stacks:** Every process has a User Stack (User Mode) and a Kernel Stack (Kernel Mode) to ensure isolation and context preservation.